

CS300 – Sabancı University Homework 4

PLEASE NOTE:

- The use of any form of AI is strictly prohibited.
- All solutions must be entirely your own work.
- No collaboration or cooperation among students is permitted.
- Submissions must be made through the SUcourse. No other method of submission will be accepted.

1. Introduction

In this homework, you will implement a Network Repair Scheduler for a communication infrastructure company. The company manages a network of stations connected by cables. Some cables are already working at the beginning, while other broken cables are reported as repair requests. Each repair request connects two stations and has an urgency value and a repair cost.

Your program will process repair operations and decide which repair request should be handled next. While doing this, your implementation must use the following data structures and algorithms:

- Disjoint Set Union to maintain connected components of the network.
- Binary Heap to select the highest-priority repair request efficiently.
- Sorting to print the current pending repair requests in priority order.

2. Program Specifications

Your program must read two text files from the current working directory:

- network.txt
- operations.txt

Your program must not ask for input from the keyboard. It must not print any prompts. It should only print the required outputs for the operations described below.

2.1 Input Files

File: network.txt:

The file network.txt describes the initial state of the network. The format is:

```
<N> <M>
<u1> <v1>
<u2> <v2>
...
<uM> <vM>
```

Where:

- N is the number of stations.
- Stations are numbered from 1 to N.
- M is the number of initially working connections.
- Each of the following M lines contains two station IDs u and v, meaning that these two stations are already connected by a working cable.

Your program must initialize a Disjoint Set Union structure with N stations and apply union operations for the initial M connections. Example:

6 1

1 2

This means there are 6 stations, and stations 1 and 2 are already connected at the beginning.

File: operations.txt :

The file operations.txt contains the operations that your program must process in order. Each line contains exactly one operation. The supported operations are:

ADD_REPAIR <repair_id> <u> <v> <urgency> <cost>
UPDATE_REPAIR <repair_id> <new_urgency> <new_cost>
PRINT_PENDING
REPAIR_NEXT
CONNECTED <u> <v>
COMPONENT_SIZE <u>
COMPONENT_COUNT
TOTAL_COST

2.2 Repair Request Priority Rule

Each repair request has a repair ID, two station endpoints, an urgency value and a repair cost. The priority of repair requests is determined by the following rules:

1. A repair request with **higher urgency** has higher priority.
2. If urgency values are equal, the repair request with **lower cost** has higher priority.
3. If both urgency and cost are equal, the repair request with the **lexicographically smaller repair ID** has higher priority.

For example, assume the following repair requests:

R1 urgency=8 cost=30

R2 urgency=10 cost=15

R3 urgency=8 cost=20

Their priority order is: R2, R3 then R1

2.3 Required Data Structures

You are required to implement the following data structures from scratch.

2.3.1 Disjoint Set Union

You must implement a Disjoint Set Union structure to maintain the connected components of the network. Your implementation must support:

find

union / unite

connected

component size

component count

Your Disjoint Set Union implementation must use path compression and union by size or union by rank.

You are **not** allowed to use BFS, DFS, adjacency matrix transitive closure, or any graph traversal method instead of Disjoint Set Union for connectivity operations.

If your program does not use Disjoint Set Union as the main connectivity structure, you may receive 0 from the relevant part of the assignment, even if the output appears correct for some test cases.

2.3.2 Binary Heap Priority Queue

You must implement your own Binary Heap to manage pending repair requests. The heap must support:

insert

extract top priority request

update priority

empty check

The heap must follow the repair request priority rule described in Section 2.2 and also you are not allowed to use: `std::priority_queue` , `std::make_heap` , `std::push_heap` , `std::pop_heap`.

Since `UPDATE_REPAIR` may change the priority of an existing repair request, your heap should maintain a mapping from repair ID to heap index. This allows efficient priority updates.

2.3.3 Sorting Pending Repairs

For the `PRINT_PENDING` operation, you must print all pending repair requests in priority order. You must implement your own sorting algorithm for this operation. Also the sorting order must be exactly the same as the repair priority rule.

2.4 Supported Operations

2.4.1 ADD_REPAIR

The ADD_REPAIR operation creates a new repair request and adds it to the set of pending repairs :

ADD_REPAIR <repair_id> <u> <v> <urgency> <cost>

Here, <repair_id> is a string identifier for the repair request, <u> and <v> are the two stations that would be connected if this repair is completed, <urgency> is the urgency level of the repair, and <cost> is the repair cost.

If the repair ID already exists, the operation must be ignored. This includes repair IDs that have already been processed or skipped. The ADD_REPAIR operation produces no output.

2.4.2 UPDATE_REPAIR

The UPDATE_REPAIR operation changes the urgency and cost of an existing pending repair request :

UPDATE_REPAIR <repair_id> <new_urgency> <new_cost>

This operation only affects repair requests that currently exist and are still pending. If the repair ID does not exist, the operation must be ignored. If the repair request has already been processed or skipped, the operation must also be ignored.

After updating the urgency and cost, the Binary Heap must be reorganized correctly. This is necessary because the priority of the repair request may increase or decrease after the update. The UPDATE_REPAIR operation produces no output.

2.4.3 PRINT_PENDING

The PRINT_PENDING operation prints all currently pending repair requests in priority order : The output must start with the following line: Pending repairs:

Then, each pending repair request must be printed on a separate line using the following format: <repair_id> <u> <v> urgency=<urgency> cost=<cost>

The order must follow the priority rule described earlier: higher urgency first, then lower cost, then lexicographically smaller repair ID. Also, if there are no pending repair requests, the output must be:

Pending repairs:

<Empty>

This operation must not remove or modify any repair request. In particular, the Binary Heap must remain unchanged after PRINT_PENDING. You must use your own sorting algorithm to produce the printed order.

2.4.4 REPAIR_NEXT

The REPAIR_NEXT operation processes the highest-priority pending repair request.

REPAIR_NEXT

If there are no pending repair requests, your program must print exactly:

No repairs pending.

Otherwise, the repair request with the highest priority is removed from the Binary Heap and processed. Suppose the selected request connects stations <u> and <v>.

If <u> and <v> are currently in different connected components, the repair is successful. In this case, your program must union the two components using Disjoint Set Union, add the repair cost to the total cost, and print:

Repaired: <repair_id>

Stations: <u>-<v>

Components: <component_count>

If <u> and <v> are already connected at the time of processing, the repair is redundant. In this case, the request is skipped, the Disjoint Set Union structure is not changed, and the repair cost is not added to the total cost. The output must be:

Skipped: <repair_id>

Stations: <u>-<v>

Components: <component_count>

A repair request that has been processed, whether successfully repaired or skipped, must no longer appear in future PRINT_PENDING outputs and cannot be updated later.

2.4.5 CONNECTED

The CONNECTED operation checks whether two stations are currently in the same connected component : CONNECTED <u> <v>

If the two stations are connected, the output must be: Connected <u> <v>: YES

Otherwise, the output must be: Connected <u> <v>: NO

Notice that this output uses a space between the two station numbers, not a hyphen. There must be exactly one space after the colon.

2.4.6 COMPONENT_SIZE

The COMPONENT_SIZE operation prints the number of stations in the connected component containing the given station : COMPONENT_SIZE <u>

The output format is: Component size of <u>: <size>

The component size must be maintained using the Disjoint Set Union structure.

2.4.7 COMPONENT_COUNT

The COMPONENT_COUNT operation prints the current number of connected components in the network. The output format is: Component count: <count>

Initially, before applying the connections in *network.txt*, the number of components is equal to the number of stations N. Each successful union operation decreases the component count by one. Skipped repairs do not change the component count.

2.4.8 TOTAL_COST

The TOTAL_COST operation prints the total cost of all successful repairs processed so far. The output format is: Total cost: <total_cost>

Only successful repairs are included in the total cost. Skipped repairs must not be included.

Important Note: During grading, the input files *network.txt* and *operations.txt* will be provided in the current working directory. Your program is expected to open these files by name and process them accordingly. Do not assume any machine-specific or absolute file path. Do not print prompts, debug messages, extra blank lines, extra spaces at the end of lines and also, Output labels must match exactly, including capitalization and punctuation.

3. Sample Runs

Below are two sample runs illustrating the expected behavior of the program. Each sample includes the contents of *network.txt*, the contents of *operations.txt*, and the corresponding program output.

Sample Run 1

network.txt:

6 1

1 2

operations.txt:

ADD_REPAIR R1 2 3 8 30

ADD_REPAIR R2 4 5 10 15

ADD_REPAIR R3 1 3 8 20

PRINT_PENDING

CONNECTED 1 3
REPAIR_NEXT
COMPONENT_COUNT
UPDATE_REPAIR R1 11 25
PRINT_PENDING
REPAIR_NEXT
CONNECTED 1 3
REPAIR_NEXT
TOTAL_COST
COMPONENT_COUNT

Expected Output:

Pending repairs:

R2 4 5 urgency=10 cost=15

R3 1 3 urgency=8 cost=20

R1 2 3 urgency=8 cost=30

Connected 1 3: NO

Repaired: R2

Stations: 4-5

Components: 4

Component count: 4

Pending repairs:

R1 2 3 urgency=11 cost=25

R3 1 3 urgency=8 cost=20

Repaired: R1

Stations: 2-3

Components: 3

Connected 1 3: YES

Skipped: R3

Stations: 1-3

Components: 3

Total cost: 40

Component count: 3

Sample Run 2

network.txt:

5 0

operations.txt:

COMPONENT_COUNT
ADD_REPAIR A 1 2 5 10
ADD_REPAIR B 2 3 5 8
ADD_REPAIR C 4 5 7 20
PRINT_PENDING
REPAIR_NEXT
COMPONENT_SIZE 4
REPAIR_NEXT
CONNECTED 1 3
REPAIR_NEXT
CONNECTED 1 3
TOTAL_COST
PRINT_PENDING
REPAIR_NEXT

Expected Output:

Component count: 5
Pending repairs:
C 4 5 urgency=7 cost=20
B 2 3 urgency=5 cost=8
A 1 2 urgency=5 cost=10
Repaired: C
Stations: 4-5
Components: 4
Component size of 4: 2
Repaired: B
Stations: 2-3
Components: 3
Connected 1 3: NO
Repaired: A
Stations: 1-2
Components: 2
Connected 1 3: YES
Total cost: 38
Pending repairs:
<Empty>
No repairs pending.

4. Submission

Your code should be submitted to SUCourse. Please follow the instructions below carefully.

Place your source files in a folder named *XXXX-NameLastname-hw4*, where *XXXX* is your student ID number. Do **not** use any Turkish characters in the folder name.

Your submission folder must contain exactly the following source files:

- source.cpp
- NetworkRepair.cpp
- NetworkRepair.h

Do **not** include executable files or unnecessary IDE/build folders such as Debug, Release, .vs, x64, or similar directories.

Compress your submission folder into a single .zip file. For example: *XXXX-NameLastname-hw4.zip*.

Before submitting, make sure that your .zip file opens correctly and reproduces your folder exactly.

Your program will be compiled and tested using the C++17 standard. Make sure that your code is compatible with C++17.

Grading

Your homework will be graded as follows:

- If your program does not use the required **Disjoint Set Union** structure as specified in this assignment, you will receive 0 points, even if your program produces correct output by using another approach.
- If your program does not implement the required **Binary Heap** priority queue from scratch and instead uses prohibited library structures such as `std::priority_queue`, `std::make_heap`, `std::push_heap`, or `std::pop_heap`, you may receive 0 points.
- We will run 10 tests on your homework using different `network.txt` and `operations.txt` files. Each correct test is worth 10 points, for a total of 100 points.
- Your output must match the required format exactly. Any deviation in formatting may cause a test to fail.

If you have any questions, please contact me at: erfan.rezaei@sabanciuniv.edu